

Learner's Semester Handbook

SAJJAN AGARWAL SCHOOL OF TECHNOLOGY

Rishihood University

**Bachelor of Technology
Computer Science & Artificial Intelligence
/
Bachelor of Technology
Computer Science & Artificial Intelligence**

Semester 5

Batch 2024 — Academic Year 2026–27

Table of Contents

Note to Learners	2
Course Structure	3
Class Timetable (Link)	3
Click Here: Master Timetable Odd Semester 2026	3
Internal Assessment: Contest Week-wise Schedule	3
Course Outline: Advanced Machine Learning	4
Course Outline: Computer Networks	8
Course Outline: Deep Learning	14
Course Outline: Modern Computer Architecture	18
Electives:	23
Course Outline: Applications Development	23
Course Outline: Embedded Systems	26
Course Outline: Game Theory	29
Course Outline: Quantum Computing	33
Attendance Policy	36
Classroom Conduct & Proxy Attendance	38
Internal Assessments & Examinations	39
Unfair Means (UFM) Policy	41
Discipline & Student Conduct	43

Note to Learners

This booklet has been prepared as a consolidated academic handbook to help learners easily understand the courses offered during the semester, academic processes, rules, regulations, and other important academic information in one place.

The academic rules, regulations, and guidelines are reviewed periodically, including at the beginning of each semester, and may be revised or updated as required. Therefore, this booklet should be used as a reference for understanding the academic framework and processes.

Learners are advised to refer to the Rishiverse Portal for the latest and officially applicable version of any academic policy, rule, regulation, or guideline. In case of any difference between the information provided in this booklet and the updated policy available on the Rishiverse Portal, the latest approved policy available on the Rishiverse Portal shall prevail.

Course Structure

Course Title	Credit
Modern Computer Architecture	4
Deep Learning	4
Advanced Machine Learning	4
Computer Network	4
Minor/Elective (Application Development/ Embedded Systems/Game Theory/ Quantum Computing)	4

Class Timetable (Link)

Click Here: [Master Timetable Odd Semester 2026](#)

Internal Assessment: Contest Week-wise Schedule

Week	Date	Remarks	Batch 2025
1	14 Aug 2026	First week	
2	21 Aug 2026	Orientation	
3	28 Aug 2026	Raksha Bandhan	
4	04 Sep 2026		Computer Networks Contest 1
5	11 Sep 2026		Deep Learning Contest 1
6	18 Sep 2026		Modern Computer Architecture Contest 1
7	25 Sep 2026		Advanced Machine Learning Contest 1
8	2 Oct 2026	Gandhi Jayanti	
9	09 Oct 2026		Elective (Game Theory + Quantum Computing)
	16 Oct 2026	Mid Term	
10	23 Oct 2026		Elective (Application Dev + Embedded System)
11	30 Oct 2026		Computer Networks Contest 2
12	06 Nov 2026	Diwali Week	
13	13 Nov 2026	Diwali Week	
14	20 Nov 2026		Deep Learning Contest 2
15	27 Nov 2026	Damru Annual Fest(28-29 Nov)	Modern Computer Architecture 2
16	04 Dec 2026	Last week	

Note: All contests will be conducted as per the schedule mentioned above. In case of any unavoidable circumstances, the schedule may be revised. Any changes to the schedule will be communicated to learners well in advance.

Course Outline: Advanced Machine Learning

1.0 Course Details

Course Title	Advanced Machine Learning		
Programme	BTech CS & AI · BTech CS & DS (Batch 2024 — NST)		
Credits	4	Course Code	CSA333
Mode	Regular Classroom	Period	11 weeks
Academic Year and Semester	2026-27 Semester 5	Category (Core/Audit)	Core
Instructor	—	LTP	2-0-4

2.0 Subject Overview

Facilitator's Vision	The goal is to help students move beyond just "importing" libraries and start understanding how machine learning actually works under the hood. By building every algorithm from scratch — from basic regressions to advanced clustering like GMMs — students will learn the logic needed to solve real-world problems, fix broken models, and handle complex data like time-series. Instead of just running code, they will gain the confidence to explain why a model works and the skill to build their own custom solutions from the ground up.
Relevance for Students	In the real world, companies don't just hire people to run code; they hire engineers who can fix models when they break. This course gives you that edge by teaching you exactly how algorithms work, so you aren't just guessing when a prediction goes wrong. You will learn how to handle professional-grade tasks like forecasting stock trends with time series or finding hidden groups in messy data with GMMs — skills that are mandatory for high-paying roles in data science and finance. Mastering these fundamentals is the only way to stand out in interviews and truly understand advanced AI later on.
Uniqueness of the Programme	Most courses today rush into Deep Learning, but this programme is different because it stays focused on mastering the fundamentals. We don't just use libraries; we build every algorithm from scratch using NumPy so you see exactly how the logic turns into code. It is also unique because it connects topics in a way that makes sense — like teaching Time Series right after Regression and showing how GMMs fix the specific mistakes that K-Means clustering makes. With two dedicated labs every week, you get double the hands-on practice compared to a normal course.
Pre-Requisites	• Good understanding of Python programming. • Good understanding of Linear Algebra, Probability, and Statistics. • Prior exposure to basic Machine Learning concepts.
Intellectual Goals & Guiding Questions	• From-Scratch Mastery: Can you write the actual math and code for a model yourself instead of just "importing" a library? • Understanding the "Why": Why does a model sometimes fail on new data, and how do we use statistical tests to prove it is actually working? • Choosing the Right Tool: When is a simple linear regression enough, and when do you really need the extra power of a Decision Tree? • Solving Complex Patterns: How do you accurately predict future trends in time-series data or find hidden groups when the data points overlap?

3.0 Educational Objectives

To meet the programme goals, this course includes "from-scratch" coding labs, statistical diagnostic tests, and hands-on projects with real-world datasets. Students will move beyond using libraries to understanding the mathematical mechanics of every algorithm they build.

By the end of this course, students will be able to:

- **Understand:** Explain the mathematical logic behind Linear and Logistic Regression. Describe how trend and seasonality affect time-series data. Understand why Gaussian Mixture Models (GMM) are used when K-Means clustering fails to handle overlapping data.
- **Apply:** Write and run optimization algorithms like Gradient Descent using only NumPy.
- **Analyze:** Break down model errors into Bias and Variance to find the best fit. Identify data problems like Multicollinearity using the Variance Inflation Factor (VIF). Inspect residuals to verify if model assumptions, like homoscedasticity, are actually met.
- **Evaluate:** Compare the performance of Logistic Regression versus Decision Trees for specific classification tasks. Evaluate clustering quality using tools like the Silhouette score and log-likelihood.
- **Create:** Design a complete machine learning pipeline that includes data cleaning, feature selection, and model tuning. Create a custom forecasting system for a real-world time-series dataset. Develop a clustering solution that can handle complex, overlapping groups using GMM.

4.0 Learning Outcomes

Domain	Outcomes
Skills	• Algorithmic Implementation: Writing core machine learning algorithms, such as Linear Regression and Gaussian Mixture Models, from scratch using only NumPy without relying on high-level libraries. • Professional Forecasting: Analyzing time-series data to identify trends and seasonality, and using ACF/PACF plots to build functional ARIMA forecasting models. • Advanced Troubleshooting: Using diagnostic tools like the Variance Inflation Factor (VIF) and residual analysis to find and fix issues like multicollinearity or biased predictions.
Knowledge	• Mathematical Logic: Understanding the math behind how the decision boundary of Logistic Regression is created. • Clustering Mechanics: Knowing exactly why Gaussian Mixture Models (GMM) are needed to handle overlapping data where K-Means clustering typically fails. • Optimization Theory: Deeply understanding how Gradient Descent navigates a loss function to find the most accurate parameters for a model.
Attitudes / Values	• Statistical Honesty: Valuing the discipline to check model assumptions and residuals rather than just blindly trusting a high accuracy score. • Scientific Skepticism: Maintaining a constant watch for "data leakage" or hidden biases that can make a model look better on paper than it is in real life. • Technical Resilience: Developing the patience and precision required to debug "from-scratch" code until the mathematical logic matches the results.

5.0 Programme Outline

Week	Content	Activity / Out-of-class	Assignments / Assessments
1	Introduction to Artificial Intelligence, Machine Learning, Deep Learning, Supervised Learning, Unsupervised Learning, Reinforcement Learning, Regression, Classification, Clustering, Machine Learning Project Life Cycle (Problem Definition, Data Collection, Exploratory Data Analysis (EDA), Data Preprocessing)	In-class prerequisite knowledge assessment.	P1: Implement EDA and Data Preprocessing from scratch.
2–3	- Machine Learning Project Life Cycle: Feature Engineering - Model Selection - Hyperparameter Tuning - Model Training - Model Testing - Model Deployment - Simple Linear Regression (SLR) using Ordinary Least Squares (OLS) - Multiple Linear Regression (MLR) using OLS - Gradient Descent: Mathematical formulation - Loss functions - Batch GD - Stochastic GD - Mini-batch GD	- Mathematics behind SLR using the closed-form solution. - Mathematical intuition behind Batch, Stochastic, and Mini-batch Gradient Descent.	P1: Implement SLR from scratch using NumPy. P2: Evaluate models using MAE, MSE, and R^2 . P1: Implement MLR from scratch. P2: Visualize the effect of learning rate on Gradient Descent convergence.
4–5	Regression Analysis & Assumptions: TSS/RSS/ESS, and Linearity assumptions. Feature Selection & Regularization: Filter/Wrapper methods and Bias-Variance Tradeoff. Dimensionality Reduction (Principal Component Analysis).	Model Diagnostics: Detecting Multicollinearity using Variance Inflation Factor (VIF). Shrinkage Methods: Geometric intuition of Ridge (L2) and Lasso (L1) regression.	P1: Residual analysis and Homoscedasticity testing. P2: Implementation of VIF to remove redundant features. P1: Comparing Lasso vs. Ridge for sparsity creation. P2: Sequential Forward Selection implementation.
6	Time Series Foundations: Trend, seasonality, noise, and ACF/PACF intuition. Forecasting Logic: AR/MA.	Forecasting Logic: AR/MA	P1: Visual pattern spotting and decomposition of real datasets. P2: Fitting basic AR and MA model to time series data.
7	Logistic Regression & Multiclass: Sigmoid function, Log Loss, and Maximum Likelihood Estimation (MLE). Classification Strategies: One-vs-Rest (OvR) and Softmax regression approach.	Classification Strategies: One-vs-Rest (OvR) and Softmax regression approach.	P1: Coding Logistic Regression from scratch via MLE. P2: Multi-class classification project (e.g., Iris/MNIST).

8–9	Decision Trees (DT): Intuition, CART algorithm, and Splitting criteria (Gini/Entropy). Random Forest.	Tree Management: Understanding overfitting, feature importance, and pruning techniques.	P1: Building a Decision Tree for classification and regression. P2: Visualizing trees and implementing Cost Complexity Pruning.
10	Clustering (K-Means): Geometric intuition, Elbow method. K-Means++ Initialization. Gaussian Mixture Models (GMM).	Limitations of K-Means: Identifying hard-clustering failures in overlapping or non-spherical data.	P1: Implementing K-Means from scratch and tracking inertia. P2: Cluster quality assessment using Silhouette score.
11	Capstone Integration: End-to-end machine learning pipeline on a single unified dataset, combining regression, time-series/PCA, classification, and clustering.	Connecting all prior techniques on one dataset; model-selection rationale across families; integrated diagnostics and evaluation.	P1: Build a complete pipeline from scratch — data cleaning, feature selection, modeling, and evaluation — on one real dataset. P2: Integration checklist and peer code review.

6.0 Evaluation Criteria

Function	% Weightage	Notes
In-class and Post-class Assignments	5%	
Contest	10%	
Projects + Viva	20%	
Mid Semester	25%	
End Semester	40%	

7.0 Readings and References

Readings	• An Introduction to Statistical Learning (ISLP) — Gareth James et al. Primary text for understanding the theory behind Regression, Classification, and Decision Trees. • Probabilistic Machine Learning: An Introduction — Kevin Murphy. Highly recommended for the intuition behind GMMs and soft clustering.
Videos / Online	• Stanford CS229: Machine Learning (Andrew Ng) — global standard for learning the math behind Classical Machine Learning Algorithms. https://www.youtube.com/playlist?list=PLoROMvovdv4rMiGQp3WXShMGgzqpfVfbU • MIT 6.390: Introduction to Machine Learning — excellent for visualizing how Linear and Logistic Regression work and understanding probability foundations. • NPTEL – Machine Learning (IIT Kharagpur/Madras) — great for the academic rigor needed for Advanced Machine Learning Algorithms https://www.youtube.com/playlist?list=PL1xHD4vteKYVpaliy295pg6_SY5qznc77
Other Resources	• Statsmodels Documentation: specifically the tsa module for implementing AR and MA models and interpreting ACF/PACF plots. • Scikit-learn User Guide: for understanding the Scikit-learn library.. • UCI Machine Learning Repository: main source for real-world datasets used in clustering and time-series labs. • Google Colab / Jupyter Notebooks: required for all lab assignments and "from-scratch" coding tasks.

Course Outline: Computer Networks

1.0 Course Details

Course Title	Computer Networks		
Programme	BTech CS & AI · BTech CS & DS (Batch 2024 — NST)		
Credits	4	Course Code	CSA223
Mode	Regular Classroom	Period	12 weeks
Academic Year and Semester	2026-27 Semester 5	Category (Core/Audit)	Core
Instructor	—	LTP	2-1-2

2.0 Subject Overview

Facilitator's Vision	In today's cloud-first world, networking is not just infrastructure plumbing — it is the foundation that every distributed AI system, microservice, and real-time application depends on. This course equips students to understand networking from first principles while simultaneously building real cloud infrastructure on AWS. Every traditional concept has a cloud equivalent, and students will experience both.
Relevance for Students	This course bridges fundamental networking theory with modern implementations and emerging technologies, preparing students for the complex networked systems they will encounter in industry.
Uniqueness of the Programme	This is a blended course: every networking concept is taught both traditionally (protocols, algorithms, packet captures) and through its AWS cloud equivalent (VPC, Route 53, ALB, Security Groups). Students use AWS Academy accounts with Learner Lab sandbox environments (\$50 budget per student), Wireshark, Cisco Packet Tracer, and Python. The top-down approach ensures students start with applications they use daily and work down to physical infrastructure, always understanding the "why" before the "what."
Pre-Requisites	• Data Structures. • Basic Python programming.
Intellectual Goals & Guiding Questions	Intellectual Goals: • Understand the layered architecture of the internet and map each layer to real applications and AWS services. • Analyze protocol behavior through packet captures (Wireshark) and cloud monitoring (VPC Flow Logs). • Design cloud network architectures using VPCs, subnets, route tables, load balancers, and DNS. • Apply security principles across both traditional (firewalls, TLS) and cloud (Security Groups, NACLs, ACM) paradigms. • Build and troubleshoot production-grade networked systems using Python, Packet Tracer, and AWS Learner Lab. Guiding Questions:

	• What happens at each network layer when you open Instagram or make a UPI payment? • How does the same CIDR subnetting math apply to both physical networks and AWS VPCs? • Why does TCP ensure reliability while UDP sacrifices it for speed, and how do ALB/NLB leverage this? • How do routing algorithms find paths across thousands of routers, and how does AWS abstract this with Route Tables? • What security threats exist at each layer, and how do cloud-native tools (Security Groups, ACM, WAF) address them?
--	--

3.0 Educational Objectives

By the end of this course, students will be able to:

1. Understand: The OSI and TCP/IP layered architectures and the function of each layer, from application protocols to physical transmission. Protocol operations including HTTP/HTTPS, DNS, TCP/UDP, IP addressing, Ethernet, and their AWS cloud equivalents (Route 53, ALB/NLB, VPC, Security Groups). Modern networking paradigms: cloud-native networking (VPCs, SDN), P2P architectures, and wireless fundamentals.
2. Apply: IP addressing, CIDR subnetting, and NAT techniques in both traditional networks and AWS VPC design. Network analysis tools: Wireshark for packet capture, VPC Flow Logs for cloud traffic, ping/traceroute/nslookup for diagnostics. Security configurations across both paradigms: firewalls and VPNs alongside Security Groups, NACLs, and ACM certificates.
3. Analyze: Network performance using delay, throughput, and congestion metrics; diagnose bottlenecks in both physical and cloud networks. Protocol behavior during connection establishment, data transfer, and error recovery using Wireshark captures and AWS monitoring. Security vulnerabilities (MITM, DNS spoofing, DDoS) and evaluate countermeasures in traditional and cloud architectures.
4. Evaluate: Network architecture designs for scalability, high availability, and security across multi-AZ AWS deployments. Protocol suitability for specific use cases (TCP vs UDP, ALB vs NLB, Route 53 routing policies). Technology trade-offs between traditional on-premises networking and cloud-native infrastructure.
5. Create: Multi-tier cloud network architectures on AWS incorporating VPCs, public/private subnets, load balancers, DNS, and TLS. Custom network configurations using subnetting, routing protocols (RIP, OSPF), Security Groups, and NACLs. Production-grade integration architectures combining traditional networking knowledge with AWS cloud services.

4.0 Learning Outcomes

Domain	Outcomes
Skills	1. Network Analysis & Cloud Design: Ability to analyze network topologies, design VPC architectures, and map traditional concepts to their AWS equivalents. 2. Protocol Implementation: Proficiency in implementing client-server applications in Python and configuring network protocols using Wireshark, Packet Tracer, and AWS console. 3. Troubleshooting: Skills to diagnose network issues using packet captures, VPC Flow Logs, traceroute, and cloud monitoring tools. 4. Security Configuration: Ability to implement security at every layer: TLS certificates via ACM, firewall rules via Security Groups/NACLs, and VPN configurations. 5. Team Collaboration: Capacity to work in teams on the semester-long cloud architecture project with phased deliverables.
Knowledge	1. OSI and TCP/IP Models: Understanding of layered architectures, their functions, and how each layer maps to AWS services. 2. IP Addressing and Cloud Networking: Knowledge of subnetting, CIDR, routing algorithms, and their application in VPC design with public/private subnets.

	3. Transport and Application Protocols: Deep understanding of TCP/UDP mechanics, HTTP, DNS, and how load balancers and Route 53 leverage these protocols. 4. Data Link and Physical Layers: Comprehension of Ethernet, WiFi, error detection, and how cloud abstracts physical infrastructure. 5. Network Tools: Familiarity with Wireshark, Packet Tracer, AWS Learner Lab, VPC Flow Logs, and Python socket programming.
Attitudes / Values	1. Problem-Solving Orientation: Enthusiasm for diagnosing real-world networking problems using both traditional tools and cloud monitoring. 2. Security-First Mindset: Recognition that security must be designed into every layer, not bolted on afterwards. 3. Curiosity: Eagerness to explore how protocols work under the hood and how cloud services abstract complexity. 4. Responsibility: Ethical approach to network administration, data privacy, and responsible use of cloud resources. 5. Adaptability: Willingness to bridge traditional networking knowledge with rapidly evolving cloud platforms.

5.0 Programme Outline

Week	Content	Activity / Out-of-class	Assignments / Assessments
1–2	How Does the Internet Actually Work? Traditional Networking: • Internet architecture: edge vs core, access networks • Packet switching vs circuit switching • Protocol layers: OSI vs TCP/IP (top-down roadmap) • Delay, loss, throughput measurement • Network devices: hubs, switches, routers, firewalls, load balancers • Topologies: star, mesh, hybrid Cloud Networking (AWS): • AWS Regions, Availability Zones, Edge Locations • VPC as a software-defined network • AWS Academy Cloud Foundations Modules 1–4	Wireshark: capture web browsing, identify DNS, TCP handshake, HTTP. traceroute to youtube.com mapped geographically. AWS: Log into Learner Lab, explore VPC console, identify default VPC components.	Quiz: Internet architecture, packet vs circuit switching, layer model.
3–4	Applications You Use Every Day: HTTP, DNS, and the Web Traditional Networking: • HTTP/1.1, HTTP/2, HTTP/3 evolution • HTTPS and TLS handshake • DNS: recursive vs iterative resolution, record types • CDNs and caching behavior • Email protocols: SMTP, POP3, IMAP • Client-server vs P2P, APIs and REST Cloud Networking (AWS):	Wireshark DNS lab. Browser DevTools: HTTP headers, caching, TLS certificate chain. AWS Lab: Build VPC and Launch a Web Server (Cloud Foundations Lab 2). Configure Route 53 hosted zone with A and CNAME records.	Assignment 1: HTTP Protocol Deep Dive — capture and annotate HTTP session, compare HTTP/1.1 vs HTTP/2, set up Route 53 DNS for EC2 instance.

	• Route 53: managed DNS, routing policies (simple, weighted, latency, failover, geolocation)		
5–6	Reliable Delivery: How TCP Makes the Internet Reliable Traditional Networking: • Transport layer multiplexing/demultiplexing (ports) • UDP: simple, fast, unreliable (gaming, video, DNS) • TCP: 3-way handshake, connection teardown • Reliable data transfer: Go-Back-N, Selective Repeat • TCP flow control (sliding window) • TCP congestion control: AIMD, slow start • TLS/SSL at the transport level Cloud Networking (AWS): • Application Load Balancer (ALB): Layer 7, path/host routing • Network Load Balancer (NLB): Layer 4, TCP/UDP • Elastic Load Balancing: health checks, target groups	Wireshark TCP lab: capture 3-way handshake, sequence numbers, retransmissions. Python: UDP Pinger (unreliable) and TCP echo server (reliable). AWS Lab: Create highly available environment with ALB + Auto Scaling (Cloud Architecting Guided Lab).	Mid-Semester Exam: Application layer protocols, DNS resolution, TCP mechanics, transport concepts, VPC basics, Route 53, ALB. (Covers Weeks 1–6)
7–8	Addressing the World: IP, Subnetting, and Cloud Network Design Traditional Networking: • IPv4 addressing: classful (historical), classless (CIDR) • Subnetting techniques: network/host bits, subnet masks • NAT and PAT • DHCP: automatic IP assignment • ICMP: ping, traceroute, error reporting • IPv6: addressing, transition mechanisms Cloud Networking (AWS): • VPC CIDR blocks: same subnetting math in the cloud • Public vs Private subnets; IGW and NAT Gateway • VPC Route Tables: local, IGW, NAT routes • AWS reserves 5 IPs per subnet	Subnetting worksheet: design addressing for 3-tier app. AWS Guided Lab: Creating a Networking Environment (VPC, subnets, IGW, NAT GW). AWS Challenge Lab: VPC Networking for Cafe (90 min). Python: subnet calculator CLI.	Assignment 2: Cloud Network Design — design VPC addressing for a startup (dev/staging/prod, 3 tiers each), implemented in Learner Lab with connectivity tests.
9–10	Finding the Path: Routing Algorithms and Network Security Traditional Networking: • Routing algorithms: Dijkstra (link state), Bellman-Ford (distance vector) • Count-to-infinity problem • Routing protocols: RIP, OSPF, BGP (basics) • Firewalls: stateful vs stateless filtering	Python: implement Dijkstra's shortest path. Packet Tracer: static and OSPF routing. AWS: configure Security Groups + NACLs for multi-tier architecture. Learner Lab: request ACM certificate, attach to ALB, HTTPS listeners.	—

	• TLS/SSL, certificate chains, symmetric/asymmetric encryption • Common attacks: MITM, DNS spoofing, DDoS • VPNs: WireGuard, IPSec, OpenVPN Cloud Networking (AWS): • Security Groups (stateful) vs NACLs (stateless): two-layer defense • AWS Certificate Manager: free TLS certs, auto-renewal • AWS WAF and Shield: DDoS protection • VPC Flow Logs: cloud-native traffic analysis	Enable VPC Flow Logs and analyze.	
11–12	The Physical Foundation and Cloud at Scale Traditional Networking: • Data Link Layer: MAC addressing, ARP, Ethernet (CSMA/CD), framing • WiFi: 802.11 standards, CSMA/CA, WPA2/WPA3 • Error detection: CRC codes, checksums • Physical Layer: transmission media, encoding, Shannon capacity Cloud Networking (AWS): • VPC Peering: direct connection between VPCs • Transit Gateway: hub-and-spoke for multiple VPCs • VPC Endpoints: private AWS service connectivity • Site-to-Site VPN and Direct Connect: hybrid cloud • CloudFront deep dive: edge caching, origin failover	AWS Lab: VPC Peering (Cloud Architecting Guided Lab). AWS Lab: CloudFront streaming. CRC and encoding exercises. Final Project presentations.	Final Project Submission and Presentation. End-Semester Exam: comprehensive (Weeks 1–12), emphasis on Weeks 7–12. Includes scenario-based Wireshark and VPC diagram questions.

*Note: Topics marked * are covered if time permits.*

6.0 Evaluation Criteria

Function	% Weightage	Notes
Contests	20%	
Projects	20%	
Mid Semester	20%	
End Semester	40%	

7.0 Readings and References

Readings	• Kurose, J. F., & Ross, K. W. (2021). Computer Networking: A Top-Down Approach (8th ed.). Pearson. • Forouzan, B. A. (2017). Data Communications and Networking (5th ed.). McGraw-Hill Education.
Videos / Online	—
Other Resources	• Coursera — Computer Networking: https://www.coursera.org/learn/computer-networking/home/module/1

Course Outline: Deep Learning

1.0 Course Details

Course Title	Deep Learning		
Programme	BTech CS & AI · BTech CS & DS (Batch 2024 — NST)		
Credits	4	Course Code	CSA332
Mode	Regular Classroom	Period	11 weeks
Academic Year and Semester	2026-27 Semester 5	Category (Core/Audit)	Core
Instructor	—	LTP	2-0-4

2.0 Subject Overview

Facilitator's Vision	To develop "AI Engineers" who possess both the mathematical intuition to understand representation limits and the engineering skills to build scalable, grounded AI products. The vision is to bridge the gap between academic research and industry standard practices.
Relevance for Students	Deep Learning powers today's AI applications in computer vision, NLP, and generative modeling. This course equips students with essential tools to innovate in research and industry.
Uniqueness of the Programme	The course integrates mathematical understanding, architectural intuition, and applied practice through structured labs. It bridges foundational neural networks with cutting-edge topics like attention, transformers, and large language models, preparing students for the modern AI landscape.
Pre-Requisites	• Mathematics II (Probability & Statistics). • Data Structures & Algorithms. • Mathematics III (Linear Algebra and Calculus). • Introductory Artificial Intelligence / Machine Learning.
Intellectual Goals & Guiding Questions	• How do linear algebra and probability shape the mechanics of deep learning models? • What optimization strategies enable deep networks to learn efficiently? • How do different architectures — CNNs, RNNs, and Transformers — model complex data? • How can regularization and autoencoders improve representation learning? • What are the limits, risks, and emerging frontiers of large-scale AI systems?

3.0 Educational Objectives

By the end of this course, students will be able to:

- **Understand:** Define the mathematical "first principles" of deep learning, including Universal Approximation Theory and Generalization gaps in high-dimensional spaces. Explain the mechanistic link between State-Space Models (SSMs/Mamba) and Linear Attention, and why they offer efficient linear-time complexity alternatives to quadratic Transformers. Comprehend the foundational theories of generative AI, specifically Forward/Backward Diffusion processes and Multimodal CLIP architectures.
- **Apply:** Design, train, and fine-tune modern architectures, including Graph Neural Networks (GNNs) for non-grid data and Latent Variable Models for synthetic data generation. Build production-grade Retrieval-Augmented Generation (RAG) systems and Agentic workflows utilizing the ReAct pattern to reduce hallucinations and ground AI responses in real data. Implement the complete LLM post-training pipeline, from Reward Modeling to Direct Preference Optimization (DPO).

- **Analyze:** Evaluate model performance using advanced benchmarks like MMLU and GPQA and analyze the trade-offs dictated by LLM Scaling Laws regarding compute, data, and parameters. Optimize models for Edge Deployment through INT4/INT8 Quantization-Aware Modular Reasoning (QAMR) and Budget Forcing for inference-time efficiency on mobile devices.
- **Evaluate:** The suitability of different architectures (e.g., CNNs, RNNs, Transformers, Autoencoders) for specific tasks in vision, speech, and language.
- **Create:** End-to-end deep learning solutions using modern frameworks, implementing and experimenting with architectures from feedforward to transformer-based models.

4.0 Learning Outcomes

Domain	Outcomes
Skills	• Design, implement, and train deep neural network architectures using modern frameworks (e.g., PyTorch, TensorFlow). • Apply optimization algorithms and regularization techniques to improve model performance and generalization. • Experiment with and fine-tune advanced architectures such as CNNs, RNNs, Autoencoders, and Transformers for real-world tasks.
Knowledge	• Explain the theoretical foundations of neural networks, activation functions, and optimization methods in deep learning. • Describe the structure and functioning of major architectures including CNNs, RNNs, Autoencoders, and Transformers. • Understand the principles behind backpropagation and gradient descent.
Attitudes / Values	• Appreciate the interplay between mathematical theory, computational design, and empirical experimentation in deep learning. • Develop curiosity and adaptability to engage with evolving AI architectures and research trends. • Value ethical responsibility, transparency, and interpretability in the development and deployment of AI systems.

5.0 Programme Outline

Week	Content	Activity / Out-of-class	Assignments / Assessments
1	Linear Algebra & Probability; Approximation Theory SVD, PCA, Bayes; Universal Approximation Theorem	NumPy matrix ops; Build XOR network; Proof worksheet on representation limits	NumPy matrix ops; Build XOR network; Proof worksheet on representation limits
2–3	Perceptron, UAT, ANN, Feedforward neural networks, Backpropagation, chain-rule, learning rate decay & Training Mechanics; Generalization Theory; Scaling Rules Activation Functions, Dropouts, Optimizers	Gradients, computational graphs; Learning in high dimensions; Optimal LR scaling	Derive gradient updates; Problem set on generalization bounds; Build an NN from scratch
4	CNN and vision algorithms Image convolution, feature extraction, architecture	Image convolution, feature extraction, architecture	CNN based digit recognition, image augmentation
5	Sequence Modeling, encoder-decoder architecture, RNNs/LSTMs	Linear Recurrent Networks & Gating; Vanishing and	Build RNN/LSTM; Construct a simple gated recurrent unit

	Linear Recurrent Networks & Gating; Vanishing and exploding gradients; Long-term dependencies	exploding gradients; Long-term dependencies	
6	Autoencoders (VAEs); GAN Basics; Similarity-Based Representation Learning Denoising and Variational AEs; GAN training; Contrastive learning (SimCLR)	Denoising and Variational AEs; GAN training; Contrastive learning (SimCLR)	VAE latent space visualization; Contrastive learning notebook
7–8	Transformers in Depth; Types of Attention and Self-Attention; Vision Transformers Multi-head attention, encoder-decoder stacks; Positional encoding	Multi-head attention, encoder-decoder stacks; Positional encoding	Build mini-transformer; Attention map visualization lab
9	Generative AI: Diffusion Models (DDPM); Multimodal Learning (CLIP), Forward/Backward noising processes; Text-to-image architectures	Forward/Backward noising processes; Text-to-image architectures	Implement a 1D Diffusion model; Fine-tune a CLIP-style model
10	LLM Tools, Scaling Laws; Post-training (RLHF, DPO, RLVR) Compute-optimal scaling; Alignment via human/AI feedback; Direct Preference Optimization	Compute-optimal scaling; Alignment via human/AI feedback; Direct Preference Optimization	Direct Preference Optimization (DPO) lab on a preference dataset; Reward-model vs. DPO comparison; Scaling-law analysis worksheet (compute/data/parameter trade-offs using MMLU or GPQA benchmark numbers)
11	AI Engineering: RAG, Quantization & Edge Deployment Vector Databases; Tool-using agents; INT4/INT8 compression (QLoRA/QAMR); 2-bit and 4-bit quantization *	Vector Databases; Tool-using agents; INT4/INT8 compression (QLoRA/QAMR)	Build production RAG pipeline; Quantization-aware fine-tuning experiment

Note: Topics marked * are covered if time permits.

6.0 Evaluation Criteria

Function	% Weightage	Notes
In Class / Post Class Assignments	5%	
Contest	10%	
Projects	25%	
Mid Semester	20%	
End Semester	40%	

7.0 Readings and References

Readings	Primary Text • Simon J.D. Prince — Understanding Deep Learning. MIT Press, 2023. Available free online at https://udlbook.github.io/udlbook Reference • Goodfellow, Bengio, and Courville — Deep Learning. MIT Press, 2016. Available free online at https://www.deeplearningbook.org/ • Géron, Aurélien — Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow (3rd ed.). O'Reilly Media, 2023. • Chollet, François — Deep Learning with Python (2nd ed.). Manning Publications, 2021.
Videos / Online	• Deep Learning Specialization — Andrew Ng, Coursera (DeepLearning.AI). • MIT OpenCourseWare: 6.S191 — Introduction to Deep Learning (MIT). • NPTEL Course: Deep Learning (CS7015) by Prof. Mitesh Khapra, IIT Madras.
Other Resources	• Course Management System (LMS/CMS): For lecture slides, lab notebooks, and assignment submissions. • Discussion Platform: Slack / Gmail. • Kaggle / Colab Integration: For coding assignments, experiments, and model training notebooks.

Course Outline: Modern Computer Architecture

1.0 Course Details

Course Title	Modern Computer Architecture		
Programme	BTech CS & AI · BTech CS & DS (Batch 2024 — NST)		
Credits	4	Course Code	CSA-222
Mode	Regular Classroom	Period	12 weeks
Academic Year and Semester	2026-27 Semester 5	Category (Core/Audit)	Core
Instructor	—	LTP	2-1-2

2.0 Subject Overview

Facilitator's Vision	This computer architecture course aims to bridge foundational principles with cutting-edge developments, creating technically proficient students who understand both the historical context and future trajectory of computing systems. By synthesizing classical computer organization with modern heterogeneous architectures, students will gain the depth necessary to navigate the rapidly evolving landscape of computing hardware.
Relevance for Students	This course provides the critical foundation to understand how computers actually work below the software layer — an essential knowledge base for: • Performance Optimization: Writing efficient code by understanding cache behavior, pipeline characteristics, and memory hierarchies. • AI/ML Engineering: Making informed hardware selection decisions for machine learning workloads. • Systems Programming: Debugging performance bottlenecks that manifest at the hardware-software interface. • Career Readiness: Participating meaningfully in technical discussions about emerging computing paradigms and architectural trade-offs.
Uniqueness of the Programme	Unlike traditional computer architecture courses that focus solely on theoretical components or outdated architectures, this programme: • Bridges Theory and Practice: Creates a direct connection from fundamental logic gates to cutting-edge heterogeneous computing systems used in modern AI applications. • Hands-On Hardware Experience: Uses microcontrollers to provide tactile experience with real silicon, not just simulators. • Industry-Relevant Focus: Emphasizes architectures actually deployed in modern systems (ARM, RISC-V).
Pre-Requisites	• Understanding of Basic Computers. • Basic Programming Concepts.
Intellectual Goals & Guiding Questions	• From Gates to Systems: How do fundamental digital logic principles scale to create the complex computing architectures that power modern applications? What are the abstraction layers that enable this scaling?

	• Performance and Bottlenecks: What architectural innovations address the inherent performance bottlenecks, and how do these innovations shape future computing paradigms? Why can't we simply "make CPUs faster"? • Specialization and Evolution: How are computer architectures evolving to accommodate specialized workloads like AI/ML, and what are the implications for software development approaches? What does domain-specific architecture mean for the future of general-purpose computing?
--	--

3.0 Educational Objectives

By the end of this course, students will be able to:

- Understand the fundamental principles of digital logic, sequential circuits, and the fetch-execute cycle that form the foundation of all computing systems, from basic processors to modern heterogeneous architectures.
- Apply architectural knowledge to write more efficient software by leveraging cache locality, vectorization opportunities, and parallel processing capabilities inherent in modern hardware designs.
- Analyze performance bottlenecks in different architectural designs, compare pipelining strategies, and evaluate the effectiveness of specialized accelerators for domain-specific workloads like machine learning.
- Evaluate architectural trade-offs between competing designs, assess the suitability of different instruction set architectures for specific applications, and critique emerging computing paradigms against established alternatives.
- Create functional processor components including ALUs, memory systems, and control units using hardware description languages, demonstrating the ability to translate theoretical concepts into working digital systems.

4.0 Learning Outcomes

Domain	Outcomes
Skills	• Hardware Design and Implementation: Design and implement digital circuits, CPU components, and memory systems using Verilog and simulation tools; translate theoretical concepts into verified hardware implementations. • Performance Analysis and Optimization: Analyze architectural bottlenecks through systematic measurement, and apply hardware knowledge to optimize software through cache-aware programming, instruction reordering, and parallel execution strategies. • System-Level Architectural Reasoning: Reason across hardware-software boundaries to make informed design decisions, understanding how high-level code maps to low-level execution and how architectural choices impact real-world performance.
Knowledge	• Foundational Computer Architecture: Master digital logic, sequential circuits, ISA design, and the fetch-execute cycle — the fundamental building blocks underlying all computing systems from microcontrollers to supercomputers. • Modern Architectural Paradigms: Understand pipelining, memory hierarchies, cache coherence, multicore parallelism, and domain-specific accelerators that define contemporary high-performance computing systems. • Hardware-Software Interface: Recognize how compiler optimizations, memory access patterns, and concurrency primitives are constrained and enabled by underlying architectural features.
Attitudes / Values	• Engineering Rigor and Precision: Appreciate systematic design methodologies and thorough verification, recognizing that hardware errors are costly and demand correctness-first thinking.

	• Curiosity and Lifelong Learning: Embrace computer architecture as a rapidly evolving field where today's cutting-edge becomes tomorrow's baseline, requiring continuous learning and adaptation. • Ethical Responsibility in Computing: Consider the societal impact of architectural choices — power consumption, accessibility, environmental sustainability — as engineering imperatives, not afterthoughts.
--	--

5.0 Programme Outline

Week	Content	Activity / Out-of-class	Assignments / Assessments
1–2	Module 1: Digital Foundations • Boolean algebra & logic gates (AND, OR, NOT, XOR) • Truth tables & logic minimization • Combinational blocks: MUX, adders • Number systems: binary, hex, two's complement • Overflow detection	Lab 1A: Verilog gates & adders implementation with testbench; GTKWave simulation. Lab 1B: 4-bit ripple-carry & carry look-ahead adders; overflow simulation.	Binary arithmetic exercises. Two's complement conversion verification.
3–4	Module 2: Sequential Circuits & Memory • SR latch, D flip-flop timing • Registers & synchronous counters • Register file design (dual read, single write) • FSM: Moore vs Mealy • SRAM vs DRAM structural comparison	Lab 2A: D flip-flop & 8-bit register implementation. Lab 2B: 4-register 32-bit register file with dual read ports; testbench validation.	FSM design: state diagrams & transition tables. Contest 1 (Modules 1–2).
5–6	Module 3: Computer Organization & ISA • Von Neumann & Harvard architectures • CISC vs RISC design philosophy • RISC-V: modular extension model • MIPS32: 32 registers, fixed 32-bit width • Instruction formats: R, I, J types • C-to-MIPS translation: if-else, loops, functions	Lab 3A: MIPS programs on MARS: array sum, max finding, branching. Lab 3B: Recursive factorial in MIPS with stack frames; disassembly comparison.	Hand-translate C functions to MIPS. Verification in MARS simulator.
7–8	Module 4: CPU Datapath & Pipelining • Fetch-decode-execute cycle • Single-cycle MIPS datapath • 5-stage pipeline: IF→ID→EX→MEM→WB • Data hazards (RAW, WAW, WAR) • Forwarding & branch prediction • CPI, IPC, Amdahl's Law	Lab 4A: MARS pipeline visualization: load-use & control hazard creation; instruction reordering. Lab 4B: Real hardware: data-dependent vs reordered loops; branch penalty measurement.	Contest 2 (Modules 3–4). Mid-Semester Exam (Modules 1–4).

9–10	Module 5: Memory Hierarchy & Cache • Memory wall & DRAM latency • Locality (temporal, spatial) • Cache organizations (direct-mapped, N-way set-associative) • Cache lookup (index/tag/offset, examples) • Replacement policies (LRU, FIFO) • Write policies (write-through vs write-back) • AMAT & multi-level analysis • Virtual memory overview: pages, page tables, TLBs	Lab 5A: Sequential vs random memory access; time array vs linked-list traversal (cycle counter); analyze cache behavior. Lab 5B: Row vs column 2D traversal; measure/explain timing; cache-blocked matrix multiply if time permits.	AMAT calculation problem set. Multi-level hierarchy analysis. Project Brief issued (end Week 10).
11–12	Module 6: Parallelism, Multicore & Modern Architectures • Dennard scaling collapse & power wall; end of single-core scaling • Multicore motivation • Cache coherence & MESI • Atomics (TAS, CAS) → mutexes • Heterogeneous computing (big.LITTLE, CPU+GPU, CPU+NPU) • ILP (superscalar, OoO, renaming) * • SIMD/vector * • GPU (SIMT, warps, blocks, HBM) * • TPU/AI hardware (systolic, chiplets) * • Spectre/Meltdown • Course wrap-up	Lab 6A: Dual-core offload: Core 0 sensor (DHT11 ISR), Core 1 compute (sliding mean/max/min); run single-core vs split; compare throughput. Lab 6B: Data race demo — 2 tasks increment shared counter (100k); show corruption; fix via mutex & atomic (<code>__atomic_fetch_add</code>); measure correctness & overhead. Lab 6C: CPU vs NumPy vs GPU matmul — Python vs NumPy (BLAS/SIMD) vs CuPy; sizes 64^2–2048^2; log-scale timing.	Contest 3 (Modules 5–6). Project evaluation (Week 12).

Note: Topics marked * in Week 11–12 are covered if time permits.

6.0 Evaluation Criteria

Function	% Weightage	Notes
Assignments / Class Participation	10%	
Mini Projects	10%	
Contests	20%	
Mid Semester	20%	
End Semester	40%	

7.0 Readings and References

Readings	• Computer Organization and Design (RISC-V or ARM/MIPS Edition) — David Patterson and John Hennessy. • Digital Design and Computer Architecture — David Money Harris and Sarah L. Harris. • Modern Processor Design: Fundamentals of Superscalar Processors — John Paul Shen and Mikko H. Lipasti. • Parallel Computer Architecture: A Hardware/Software Approach — David Culler, J.P. Singh, and Anoop Gupta.
Videos / Online	• Digital Design and Computer Architecture — L1: Intro: Fundamentals, Transistors, Gates (Spring 2025): https://www.youtube.com/watch?v=ubhxKNIOIRg&list=PL5Q2soXY2Zi9Eo29LMgKVcaydS7V1zZW3 • Introduction to the book: Basic Computer Architecture: https://www.youtube.com/watch?v=uCO_rNJzXcDU&list=PL1iLu2CSC9EWAo0ysorNI_nebwF6Rwkr0&index=1 • Advanced Computer Architecture: https://www.youtube.com/watch?v=f2WuXJEFi4A&list=PL1iLu2CSC9EWZMIh4_V5dGroMAWA84Lmz
Other Resources	• The Elements of Computing Systems (2nd ed.) — Naom Nisan and Schocken (Nand2Tetris): https://www.nand2tetris.org/ • CPULator (online simulator): https://cpulator.01xz.net/ • MARS MIPS Simulator: https://dpetersanderson.github.io/

Electives:

Course Outline: Applications Development

1.0 Course Details

Course Title	Applications Development		
Programme / Applicable To	Elective — BTech CS & AI · BTech CS & DS (Batch 2024 — NST) (Available in Semester 3 and Semester 5 to students not opting for a Minor)		
Credits	4	Course Code	CSA327
Mode	Regular Classroom	Period	12 weeks
Academic Year and Semester	2026-27 Semester 3 (Year 2) and Semester 5 (Year 3)	Category	Elective
Instructor	—	LTP	2-1-2

2.0 Subject Overview

Facilitator's Vision	To equip students with the technical and design skills to build modern applications across different platforms, with a strong emphasis on hands-on learning.
Relevance for Students	Application development is among the most in-demand software skills today. This course enables students to design and build real-world applications across multiple platforms and environments using modern development approaches.
Uniqueness of the Programme	Unlike platform-specific courses, this programme introduces students to core concepts of application development that apply across multiple domains, including terminal, mobile, desktop, and distributed applications.
Pre-Requisites	• Understanding of any modern programming language. • Problem-solving and logical thinking. • Familiarity with user interface development basics.
Intellectual Goals & Guiding Questions	• How do application design principles vary across different platforms and environments? • How can one design, build, and manage interactive applications effectively? • How are external services, data, and system capabilities integrated into applications? • What are the key practices for testing, debugging, securing, and deploying applications?

3.0 Educational Objectives

By the end of this course, students will be able to:

- Understand the fundamentals of application development across different platforms.
- Design and build interactive applications using appropriate architectural and interface patterns.
- Integrate external services, data, and system capabilities into applications.
- Implement user interactions, data flow, and persistence in applications.
- Test, debug, and deploy application solutions using modern development practices.

4.0 Learning Outcomes

Domain	Outcomes
Skills	• Build terminal-based, mobile, desktop, and other interactive applications for real-world use cases. • Create application interfaces, flows, and features based on user and system needs. • Develop applications that interact with data, external services, and platform capabilities. • Test, debug, and improve applications for functionality, reliability, and user experience. • Design and develop complete working applications from idea to execution.
Knowledge	• Fundamentals of building applications across terminal, mobile, desktop, and other emerging environments. • Principles of application design, interaction flow, and user experience. • Concepts involved in integrating data, services, and system capabilities into applications.
Attitudes / Values	• Developing a builder mindset through hands-on problem solving and experimentation. • Focusing on usability, reliability, and real-world relevance while creating applications. • Writing clean, modular, and maintainable solutions during application development.

5.0 Programme Outline

Week	Content	Activity / Out-of-class	Assignments / Assessments
0–2	Introduction to application development Understanding different types of applications Terminal-based application development How applications like Claude Code and ChatGPT terminal are built	Application 1 Application 2	Theory and Coding based MCQs Coding Assessments
3–4	Mobile application development Building applications for communication and social interaction Exploring and building applications such as WhatsApp and Instagram Understanding user journeys in mobile experiences	Application 1 Application 2	Theory and Coding based MCQs Coding Assessments
5–6	Extending mobile application development into content, media, and everyday utility Exploring and building applications such as Spotify and similar digital products Understanding personalization and user engagement Creating seamless digital experiences	Application 1 Application 2	Theory and Coding based MCQs Coding Assessments
7–8	Desktop application development Building applications used in professional workflows How can you build your own Athena-like application? Understanding applications built for real-world use cases	Application 1 Application 2	Theory and Coding based MCQs Coding Assessments
9–10	Designing applications for real users and real problems Understanding how applications interact with systems and services Exploring intelligent and workflow-based applications	Application 1 Application 2	Theory and Coding based MCQs Coding Assessments

11–12	Interactive and immersive applications How can we make an Xbox app or game? How can you make your own game? How can we make a VR-based application?	Application 1 Application 2	Theory and Coding based MCQs Coding Assessments
-------	--	--------------------------------	--

6.0 Evaluation Criteria

Function	% Weightage	Notes
Assignments / Class Participation	10%	
Contest	20%	
Capstone Project	10%	
Mid-Semester	20%	
End Semester	40%	

7.0 Readings and References

Readings	Documentation https://www.electronjs.org/ https://reactnative.dev/ https://nodejs.org/docs/latest/api/ https://www.geeksforgeeks.org/operating-systems/what-is-command-line-interface-cli/
Videos / Online	—
Other Resources	https://www.youtube.com/watch?v=KVZ-P-Zl6W4 (Introducing React Native)

Course Outline: Embedded Systems

1.0 Course Details

Course Title	Embedded Systems		
Programme / Applicable To	Elective — B.Tech CS & AI · B.Tech CS & DS (Batch 2024 — NST) (Available in Semester 3 and Semester 5 to students not opting for a Minor)		
Credits	4	Course Code	CSA341
Mode	Regular Classroom	Period	12 weeks
Academic Year and Semester	2026-27 Semester 3 (Year 2) and Semester 5 (Year 3)	Category	Elective
Instructor	—	LTP	2-1-2

2.0 Subject Overview

Facilitator's Vision	This course provides a comprehensive understanding of embedded systems, focusing on both hardware and software aspects. With a hands-on approach using ESP32/Arduino or STM boards, students will learn to design embedded systems and develop real-time applications.
Relevance for Students	Students will be introduced to Embedded C, ARM architecture, and real-time operating systems (RTOS). By integrating software and hardware concepts, this course will provide a gateway into firmware engineering, which is an emerging and highly demanded field, especially in AI/ML and IoT-driven domains.
Uniqueness of the Programme	The course emphasizes practical learning, where students will directly engage in designing and programming embedded systems. The integration of hardware with real-time software allows students to bridge their theoretical knowledge with practical, employable skills.
Pre-Requisites	Basic knowledge of C programming, Computer Architecture, and Basic Electronics.
Intellectual Goals & Guiding Questions	• How does embedded system design integrate hardware with software to solve real-world problems? • What are the challenges in designing embedded systems that work reliably in real-time environments? • How can we optimize embedded systems for performance, power, and size using various hardware/software techniques?

3.0 Educational Objective

By the end of this course, students will be able to:

- Understand embedded system design principles, focusing on real-time constraints and system-level programming.
- Program microcontrollers and ARM-based embedded systems in Embedded C and assembly.
- Interface embedded systems with sensors, actuators, and other peripheral devices using various communication protocols (SPI, I2C, UART).
- Develop real-time software with an understanding of RTOS concepts, task management, and inter-process communication.
- Build and test small embedded systems for applications such as process control, IoT devices, and robotics.

4.0 Learning Outcomes

Domain	Outcomes
Skills	• Embedded C programming: Ability to write optimized code for embedded systems, including task management in RTOS. • Hardware interfacing: Designing systems that interface with sensors, actuators, and communication protocols. • System optimization: Ability to optimize embedded systems for speed, power consumption, and memory usage.
Knowledge	• Embedded architecture: Understanding of ARM architecture and microcontroller programming. • RTOS: Familiarity with real-time operating systems, multitasking, and inter-task communication. • Embedded software development: Deep understanding of software tools, emulators, and debugging techniques.
Attitudes / Values	• Precision in design: Emphasis on error-free and efficient embedded system development. • Curiosity for hardware and software integration: A commitment to learning both the hardware and firmware aspects of embedded systems. • Ethical responsibility: Understanding the ethical considerations in embedded system applications, especially in IoT and safety-critical systems.

5.0 Programme Outline

Week	Content	Activity / Out-of-class	Assignments / Assessments
1–2	Introduction to Embedded Systems: What is an Embedded System and why does it matter? Course expectations, instructor introductions Grading structure and lab overview Basic concepts, hardware and software components, sensors, actuators, memory, and communication interfaces.	Decode-Your-Board: Create a pin & register map cheat-sheet from the ESP32-S3 datasheet; identify GPIO, timer, ADC, UART registers.	HW: One-page datasheet autopsy for a different MCU (Cortex-M/AVR). Map GPIO+timer registers and compare to ESP board.
3–4	Embedded C for Hardware: Bitfields/endianness/alignment, memory barriers, driver layering (register → HAL → app), simple unit tests on target. Interrupts & Timers: Timer/counter, capture/compare, ISR contract, latency/jitter, watchdog.	Time-Correct LED + Button: Build debounced button → timer ISR → task callback, measure jitter under load, report latency.	Programming assignment on ARM architecture.
5–6	Interfacing & Communication Protocols: UART framing, I ² C transactions, SPI modes, memory/device-mapped I/O patterns, arbitration basics. Sensors/Actuators: ADC sampling theory, quantization, simple filters, PWM for motor/LED, calibration.	Interface a temperature sensor to ESP32 board: Write a minimal device driver from its datasheet — init sequence, read/write registers, error handling.	Lab report on sensor interfacing.

7–8	Real-Time Operating Systems (RTOS): RTOS basics, multitasking, task scheduling, and inter-task communication. Review Session: Review of key concepts from weeks 1–6 and practice problems and solutions.	RTOS Implementation: Implementing simple RTOS tasks and scheduling on ESP32 boards.	Quiz on RTOS concepts and task management.
9–10	Optimization & Codesign: Profiling & cycle counting, register allocation, instruction-level effects, HW/SW partitioning, FSM to driver mapping, memory technologies & simple interfacing.	Measure–Optimize–Validate: Start with a naive sensor pipeline, profile, then optimize buffering, DMA-friendly bursts, precomputed tables; measure before/after throughput & power.	Optimization Assignment: What's in ISR, what's in task, what's precomputed — include FSM diagram.
11–12	IoT Applications: IoT architecture, wireless communication (Bluetooth, WiFi, Zigbee), and cloud integration. Review Session: Review of key concepts from weeks 1–6.	Productization Drill: Define requirements → pin/clock plan → driver list → RT schedule & priorities → measurement plan → demo.	Project Submission: Projects based on ESP32 board with the concepts learnt throughout this course.

6.0 Evaluation Criteria

Function	% Weightage	Notes
Assignments / Class Participation	10%	
Mini Projects	20%	
Contests	10%	
Mid Semester	20%	
End Semester	40%	

7.0 Readings and References

Readings	1. Embedded System Design: A Unified Hardware/Software Introduction — Frank Vahid and Tony Givargis 2. An Embedded Software Primer — David E. Simon 3. Embedded Microcomputer System Real Time Interfacing — Jonathan Valvano
Videos / Online	—
Other Resources	—

Course Outline: Game Theory

1.0 Course Details

Course Title	Game Theory		
Programme / Applicable To	Elective — BTech CS & AI · BTech CS & DS (Batch 2024 — NST) (Available in Semester 3 and Semester 5 to students not opting for a Minor)		
Credits	4	Course Code	CSA335
Mode	Regular Classroom	Period	12 weeks
Academic Year and Semester	2026-27 Semester 3 (Year 2) and Semester 5 (Year 3)	Category	Elective
Instructor	—	LTP	2-1-2

2.0 Subject Overview

Facilitator's Vision	To build strategic thinking and mathematical maturity in students — enabling them to model, reason about, and solve problems where outcomes depend on the decisions of multiple interacting agents. The goal is to bridge classical game theory with its modern applications in algorithmic systems, machine learning, and data-driven decision-making.
Relevance for Students	Game theory underpins a wide range of fields critical to CS & AI students: multi-agent reinforcement learning, mechanism design, auction bidding systems, social network analysis, and competitive strategy. This course makes students literate in a language spoken across economics, computer science, and political science.
Uniqueness of the Programme	This course uniquely integrates classical game theory with its computational counterpart (algorithmic game theory), making it directly relevant to CS students. Students will not only prove and derive solution concepts but also simulate strategic interactions programmatically — building intuition through code, experiments, and a capstone tournament.
Pre-Requisites	• Mathematics: Probability & Statistics, Linear Algebra basics, Discrete Mathematics. • Programming: Python (NumPy, basic data structures). • Prior exposure to proofs or formal reasoning is an advantage.
Intellectual Goals & Guiding Questions	1. Model: How can strategic interactions between rational agents be represented as formal mathematical objects (games)? 2. Solve: What are the principled solution concepts (Nash Equilibrium, dominant strategies, subgame perfection) and under what conditions do they apply? 3. Design: How can a game itself be designed — through mechanism design and auction theory — to produce desirable outcomes? 4. Compute: How do computational constraints alter the nature of equilibrium concepts, and how can we simulate and find equilibria algorithmically?

3.0 Educational Objectives

By the end of this course, students will be able to:

- Understand core game-theoretic models — normal form, extensive form, Bayesian games, and repeated games — and the mathematical reasoning underlying solution concepts like Nash Equilibrium and backward induction.
- Apply solution concepts to solve strategic problems in economics, computer science, and social choice, including auction design, voting mechanisms, and network formation.
- Analyze multi-agent interactions by modeling them as formal games, computing equilibria, and interpreting the implications for system design and agent behavior.
- Evaluate the validity and limitations of equilibrium predictions, including issues of multiplicity, computational hardness, and bounded rationality.
- Create a fully functional simulation of a strategic environment (e.g., auction, tournament, negotiation protocol) as a capstone project demonstrating applied game-theoretic reasoning.

4.0 Learning Outcomes

Domain	Outcomes
Skills	1. Formulate real-world strategic situations as normal or extensive form games. 2. Compute Nash Equilibria, dominant strategies, and subgame perfect equilibria by hand and algorithmically. 3. Design simple mechanisms (auctions, voting rules) and reason about their properties (efficiency, incentive-compatibility).
Knowledge	1. Master the mathematical foundations: normal form, extensive form, mixed strategies, Bayesian games, and repeated games. 2. Understand core solution concepts and their assumptions, including Nash Equilibrium, backward induction, and the Folk Theorem. 3. Identify the connections between game theory and fields including algorithmic mechanism design, ML (GANs, MARL), and social choice.
Attitudes / Values	1. Develop strategic thinking — the ability to reason from the perspective of other agents. 2. Appreciate the tension between individual rationality and collective welfare (e.g., Prisoner's Dilemma, Tragedy of the Commons). 3. Build intellectual humility about the limits of rationality assumptions and classical equilibrium concepts.

5.0 Programme Outline

Week	Content	Activity / Out-of-class	Assignments / Assessments
1	L1: Course Overview & Motivation — Why Game Theory? L2: Representing Games — Normal Form, Payoff Matrices	In-Class Activity: 2x2 game examples (Battle of the Sexes, Matching Pennies). Identify dominant strategies by inspection.	Problem Set 1: Constructing payoff matrices for given scenarios.
2	L3: Dominant Strategies & Iterated Elimination (IESDS) L4: Nash Equilibrium — Definition and Existence	Workshop: IESDS practice. Solve 3x3 and 4x4 games. Discuss what IESDS cannot eliminate.	Quiz 1: Normal form games, dominance, and basic Nash Equilibrium.

3	L5: Board Game Analysis I: Early-Game Mechanics & Dominance * L6: Mixed Strategy Nash Equilibrium	Interactive Session: Analyzing simple board/card games like Sequence using pure NE. Python Lab: Simulate the Matching Pennies game.	Problem Set 2: Compute mixed strategy NE for 2×2 and 2×3 games.
4	L7: Computing Mixed Strategy NE — Indifference Conditions L8: Extensive Form Games — Trees, Information Sets	Interactive: Build and solve game trees for parlor games (Nim, Ultimatum Game).	Assignment 1: Model a 3-stage bargaining game and solve by backward induction.
5	L9: Backward Induction & Subgame Perfect Equilibrium L10: Repeated Games — Finitely and Infinitely Repeated	Case Study: Repeated Prisoner's Dilemma — why can cooperation emerge? Discuss Tit-for-Tat.	Problem Set 3: Identify sustainable outcomes under infinite repetition using discount factor δ .
6	L11: Folk Theorem & Cooperation in Repeated Games * L12: Bayesian (Incomplete Information) Games	Python Lab: Implement a Bayesian game simulation — first-price sealed-bid auction with private valuations.	Quiz 2: Extensive form, backward induction, and intro to Bayesian games.
7	L13: Bayes-Nash Equilibrium & Applications L14: Auction Theory — First Price, Second Price (Vickrey)	Simulation: Run a live classroom auction. Compare bidding strategies and revenues across formats.	Problem Set 4: Derive optimal bidding strategies for first- and second-price auctions.
8	L15: Board Game Analysis II: Hidden Info & Bluffing * L16: Revenue Equivalence Theorem	Workshop: Analyzing strategic depth in games like Poker, Azul, Catan or Coup using Bayesian concepts.	Mid-Term Exam (Covering Weeks 1–7)
9	L17: Mechanism Design — Goals, Revelation Principle L18: Incentive Compatibility & Individual Rationality	Workshop: Design a simple mechanism for allocating a shared resource. Verify IC and IR properties.	Problem Set 5: Verify incentive compatibility for a given direct revelation mechanism.
10	L19: VCG Mechanisms — Efficiency in Combinatorial Settings L20: Algorithmic Game Theory Introduction & Price of Anarchy	Python Lab: Implement a VCG allocation and payment rule for a small combinatorial auction.	Quiz 3: Mechanism design and VCG.
11	L21: Network Games — Routing, Congestion, Braess's Paradox * L22: Cooperative Game Theory — Coalitions, Core, Shapley Value	Simulation: Model Braess's Paradox in a small network. Observe emergent inefficiency.	Problem Set 6: Compute Shapley values for a 3-player cooperative game.

12	L23: Game Theory in AI — GANs, MARL, and Multi-Agent Systems L24: Capstone Presentations & Course Wrap-Up	Capstone Project Demo: Student groups present their simulated strategic environments.	Assignment 2 (Capstone): Build and present a programmatic simulation of a strategic environment.
----	--	---	--

Note: Lectures marked with an asterisk () in the outline are "Buffer & Enrichment" sessions. These topics will be covered only if the core conceptual foundations are fully mastered within the scheduled timeline. If the class requires more time for simulations, board game analysis, or capstone project development, these lectures may be replaced with additional workshop sessions.*

6.0 Evaluation Criteria

Function	% Weightage	Notes
Contest	10%	
Project	20%	
Viva / Presentation	10%	Conceptual questioning, real-world applications, in-person Viva
Mid Semester	20%	Mid-term exam
End Semester	40%	Final Exam

7.0 Readings and References

Readings	Required 1. An Introduction to Game Theory — Martin Osborne (Free at economics.utoronto.ca/osborne/igt/) 2. Game Theory, Alive — Karlin & Peres (AMS, 2017 — rigorous CS-oriented coverage of algorithmic game theory) Suggested 1. The Art of Strategy — Avinash Dixit & Barry Nalebuff - Multiagent Systems — Shoham & Leyton-Brown (Free at masfoundations.org) 2. Games of Strategy — Dixit, Skeath & McAdams 3. A Brief Introduction to the Basics of Game Theory — Matthew O. Jackson (Free on SSRN)
Videos / Online	Required 1. Open Yale: ECON 159 — Game Theory (Prof. Ben Polak) — Free on YouTube 2. Stanford / Coursera: Game Theory (Jackson, Shoham, Leyton-Brown) — Free to audit on Coursera Suggested 1. Game Theory 101 by William Spaniel — YouTube (search "Game Theory 101 Spaniel") 2. Stanford CS109: Probability for Computer Scientists — Free on YouTube 3. 3Blue1Brown — Visual intuition on relevant math (probability, linear algebra)
Other Resources	Required 1. Python Libraries: NumPy (numpy.org/doc/), SciPy (scipy.org), NetworkX (networkx.org) — for simulation labs. 2. Axelrod's Tournament Papers (1980, 1981) — on iterated Prisoner's Dilemma; freely available online. Assigned reading for Week 5. Suggested 1. Seeing Theory — seeing-theory.brown.edu — Interactive visual explanations for probability and strategy concepts. 2. Axelrod Python Library — github.com/Axelrod-Python/Axelrod — Ready-to-use framework for running iterated Prisoner's Dilemma tournaments.

Course Outline: Quantum Computing

1.0 Course Details

Course Title	Quantum Computing		
Programme / Applicable To	Elective — BTech CS & AI · BTech CS & DS (Batch 2024 — NST)		
Credits	4	Course Code	CSA324
Mode	Regular	Period	14 weeks
Academic Year and Semester	2026-27 Semester 5 (Year 3) only	Category	Elective
Instructor	—	LTP	2-1-2

2.0 Subject Overview

Facilitator's Vision	To empower students with a conceptual and practical grasp of quantum computing, helping them bridge theory and implementation through visualization, coding, and algorithmic thinking.
Relevance for Students	This course prepares students for the emerging quantum era by developing their understanding of quantum logic, uncertainty, and computational advantage in real-world contexts such as cryptography, optimization, and machine learning.
Uniqueness of the Programme	Unlike abstract or purely theoretical quantum courses, this programme emphasizes visual and interactive learning through Qiskit, case studies of quantum protocols, and core algorithm demonstrations — building a bridge between quantum theory and practical insight.
Pre-Requisites	• High-school level understanding of linear algebra (Mathematics 3) and complex numbers. • Familiarity with Python is recommended for hands-on Qiskit sessions (PSP).
Intellectual Goals & Guiding Questions	1. What makes quantum computation fundamentally different from classical computation? 2. How do we represent and manipulate quantum information using gates and circuits? 3. What are key quantum phenomena such as superposition and entanglement, and how do they enable new algorithms? 4. How can we simulate and visualize quantum processes using tools like Qiskit? 5. What is the role of error correction and fault tolerance in real-world quantum devices?

3.0 Educational Objectives

By the end of this course, students will be able to:

- Understand the fundamental principles of quantum computing including qubits, superposition, entanglement, and quantum gates.
- Apply quantum logic operations and circuit design principles to simulate and implement quantum algorithms using tools like Qiskit as well as IBM quantum machine.
- Analyze quantum circuits and algorithmic behavior to explain computational advantages over classical methods.

- Evaluate the role of quantum phenomena such as interference and entanglement in real-world protocols like teleportation, cryptography, and error correction.
- Create and simulate quantum circuits for algorithms such as Grover's and Deutsch–Jozsa, and interpret their output to solve computational problems.

4.0 Learning Outcomes

Domain	Outcomes
Skills	• Design and analyze quantum circuits using single- and multi-qubit gates. • Simulate and visualize quantum states and algorithms using Qiskit and Google Colab. • Apply quantum computing concepts to real-world protocols like teleportation, superdense coding, and quantum search.
Knowledge	• Core concepts of quantum mechanics relevant to computation: superposition, entanglement, and measurement. • Structure and functionality of quantum gates and algorithms. • Foundations of quantum error correction, noise models, and fault tolerance.
Attitudes / Values	• Appreciation for the paradigm shift from classical to quantum computation. • Curiosity and open-mindedness toward abstract and counterintuitive phenomena. • Ethical reflection on the potential impact and limitations of quantum technologies in society.

5.0 Programme Outline

Week	Content	Activity / Out-of-class	Assignments / Assessments
1	Introduction to Quantum Computing and Qubits	Interactive lecture on classical vs quantum, Bloch sphere demo	Concept check quiz on classical vs quantum bits
2	Quantum States, Superposition, Measurement	Qiskit demo: initial states, superposition	Lab: Simulating single-qubit states
3–4	Bloch Sphere, Quantum Gates and Circuits (X, H, Z, RX, RZ)	Hands-on: Qiskit circuit construction, visualizations	Assignment: Build and interpret simple quantum circuits
5	Multi-Qubit Systems & Entanglement	Bell state creation, circuit demos	Exercise: Build a Bell circuit and verify entanglement
6	Quantum Algorithms I: Superdense Coding, Quantum Teleportation	Step-by-step protocol simulations in Qiskit	Assignment: Teleportation protocol implementation
7	Quantum Algorithms II: Deutsch-Jozsa	Algorithm walkthrough and simulation	Mini-project: Deutsch-Jozsa problem instances
8	Quantum Fourier Transform & Grover's Algorithm	Visualizing QFT steps, amplitude amplification	Assignment: Simulate QFT and apply Grover's algorithm
9	Introduction to Shor's Algorithm	Modular arithmetic primer, high-level Shor overview	Quiz: Quantum algorithm components
10	Quantum Noise Overview	Bit flip & phase flip code simulation in Qiskit	Worksheet: Identify and mitigate noise effects

11	Physical Implementation of Qubits Processor vs QPU, Different physical systems and DiVencenzo Criteria	Worksheet about different hardware	—
12–14	Quantum Machine Learning Overview	Case studies, IBM Q & real device execution	Final research project: Simulate a quantum solution to a real-world task

6.0 Evaluation Criteria

Function	% Weightage	Notes
Contest	10%	
Coding Assignments / Research Paper Writing	25%	Given student exposure with research paper / code recreation
Mid Semester	25%	Mid-term exam
End Semester	40%	Final Exam

7.0 Readings and References

Readings	1. Quantum Computation and Quantum Information — Nielsen & Chuang 2. Quantum Computing for Computer Scientists — Yanofsky & Mannucci 3. Quantum Computing: A Gentle Introduction — Eleanor Rieffel and Wolfgang Polak
Videos / Online	1. Introduction to Quantum Computing (YouTube): https://youtu.be/2SPjEA-4IKk?si=gqOtiMeOp1oeplnY 2. IBM Quantum Learning: https://learning.quantum.ibm.com/
Other Resources	1. IBM Quantum Platform: https://quantum.ibm.com/

Attendance Policy

Regular attendance is the foundation of learning. Missing classes not only affects your grades but also your eligibility to sit for exams.

1.1 Minimum Attendance Requirements

Attendance Level	What It Means
100%	Expected from every student — this is the goal
75% or above	Minimum required to sit for exams in that subject
60% – 74%	Allowed only for official university duties (projects, sports competitions, conferences, fests) with prior written approval from the competent authorities
50% – 59%	Allowed only for serious medical emergencies, or an approved startup under Venture Studio guidance, with prior approval
Below 50%	Not acceptable under any circumstances — automatically debarred in that subject

Simple Rule to Remember: You need at least 75% attendance in every course to sit for the end-semester exam. The 25% gap is there for **real emergencies — not casual skipping**.

1.2 How Attendance is Calculated

Attendance is calculated separately for each course, with attendance from both theory and practical sessions being considered. The attendance for lecture and practical (lab) sessions is combined, and the average of both is taken as the final attendance percentage for the course.

Any permission, attendance leeway request, or attendance-related matter concerning a course offered by another School shall be addressed to and managed by the respective School offering the course.

1.3 Permitted Absences & Leeway Provisions

The university provides leeway in specific situations. This **does NOT mean you are marked present** — it means your final attendance percentage will be considered after accounting for approved absences.

Your Attendance	Reason Allowed	What You Must Do
75% – 100%	Minor illness (fever, cold, cough) or personal commitments including volunteering	Email your Program Office within a reasonable time. No prior formal permission is required, provided your overall attendance stays at or above 75%.
60% – 74%	Representing RU in approved projects, sports competitions, conferences, or university-approved events/fests	Get PRIOR written approval from Program Lead AND faculty coordinator BEFORE the event. Approval may be denied if it clashes with critical academic activities (e.g., final exams). Approved absences still count toward the leeway — attendance must stay at or above 60%.

50% – 59%	Surgery, accident, fracture, contagious disease, family crisis, or an approved startup under Venture Studio guidance	For medical: submit an MBBS doctor's certificate with diagnosis, registration number, and minimum 15-day bed rest. Inform your Program Office within 2 days; get documents verified by the Rishihood Infirmary within 3 working days of your return to campus; submit the Infirmary-approved document to the Program Office within 5 working days of your return.
Below 50%	No exceptions permitted	No leeway available. Automatic Fail in that course.

1.4 Special Leeway Provisions

Medical Absence (50–59% range)

- Certificate from a registered MBBS doctor is mandatory
- Doctor's certificate must include: diagnosis, recommended bed rest (minimum 15 days), and the doctor's registration number
- Inform your Program Office within 2 days of the absence starting (you or a family member may do this)
- Get the supporting documents verified by the Doctor at the Rishihood Infirmary within 3 working days of your arrival back on campus — only Infirmary-verified medical documents are eligible for medical leeway
- Submit the Infirmary-approved medical document to the Program Office within 5 working days of your arrival back on campus. Make-up assessments missed during medical leave may be requested, subject to faculty and Program Office approval

Program Office Email id: csai.leadoffice@nst.rishihood.edu.in

Startup / Venture Studio (50–59% range)

- Must take prior approval from Venture Studio and Program Leads at the time of selection
- Approval must come before you start missing classes
- The Venture Studio will review your work as per its own mechanism and inform your Program Office of the attendance leeway before the final attendance submission deadline

1.5 Consequences of Low Attendance

Attendance Status	Consequence
Below 75% without approved leeway	Debarred from the end-semester exam for that course
Below 50% in any course	Automatic Fail — no leeway can change this
15 consecutive days absent without any notice	University assumes voluntary withdrawal from the program; you will be removed from class rolls

Submitting fake medical documents	Debarred from the entire semester (all courses); further disciplinary action possible
Disciplinary suspension period	Days of suspension count as unexcused absences — can push you below 75% threshold

Voluntary Withdrawal Warning: If you go missing for 15 consecutive days without informing the university, you will be removed from class rolls. Re-admission is not guaranteed and requires fees.

Key Rules to Remember

- 75% is the absolute minimum in every course to appear for exams
- 100% attendance is expected; the 25% leeway is for genuine emergencies, not routine skipping
- Always get written approval BEFORE missing classes when in the 60–74% zone
- Medical certificates must meet specific requirements — MBBS doctor's registration number, diagnosis, and minimum 15-day bed rest are all mandatory, and must be verified by the Rishihood Infirmary
- Spot an attendance discrepancy on Rishiverse? Report it to your Program Office and faculty within 2 working days of the class — requests made after that window will not be entertained
- Personal requests or parental requests will not be entertained beyond this policy

Classroom Conduct & Proxy Attendance

Being physically present in class is not enough. How you conduct yourself in the classroom matters too.

2.1 Classroom Expectations

Every student is expected to:

- Arrive on time and be seated before class begins
- Wear appropriate and decent attire and footwear in the classroom and on campus.
- Maintain silence and avoid side conversations
- Keep mobile phones on silent or switched off
- Refrain from using phones for non-academic purposes during class
- Follow all instructions given by faculty and staff
- Sit attentively and participate respectfully
- Raise questions or concerns politely and at appropriate times

Zero Tolerance in Class: *Aggressive behaviour, verbal abuse, disrupting class deliberately, or threatening faculty/staff will result in immediate disciplinary action.*

2.2 What Is Proxy Attendance?

Proxy attendance is any act of marking attendance dishonestly. There are two types:

Type of Proxy	Description
Proxy for yourself	You marked your attendance and then leave the class without proper approval from faculty and approving authority. This act will come under proxy attendance.

2.3 Consequences of Proxy Attendance (Internal Policy)

Proxy attendance is treated as academic dishonesty. Consequences escalate with each offence:

Offence	Consequence	Impact
1st Offence	Marked absent for all academic activities for the past 2 days	Retroactive loss of 2 days of attendance
2nd Offence	Final marks for that subject reduced by 10 marks at end of term	Irrespective of prior cases — marks deducted from final score
3rd Offence	Zero marks in Internal marks for that subject — forced reappearance or course repeat	Subject to reattempt fees; transcript shows failure

Internal Assessments & Examinations

All assessments — from contests to the end-semester exam — must be attended in person. There are no online alternatives..

3.1 Types of Assessments

Assessment Type	Description
Weekly Contest	Short recurring assessments held through the semester, testing recent topics
Mid-Term Examination	Formal exam held mid-semester covering syllabus taught up to that point
End-Semester Examination	Final comprehensive exam held at the end of the semester; conducted under COE Office supervision
Practical Evaluations	Hands-on lab or studio assessments as part of practical courses
Re-Contest / Re-Exam	Not a separate examination type, but a one-time opportunity to appear for a contest or mid-term assessment missed due to a valid reason, subject to prior approval .

Note: For any contest or mid-term missed before or up to the mid-term period, one re-examination opportunity will be provided as per the schedule announced by the University.

For contests missed after the mid-term, marks may be appropriated as per the applicable academic guidelines instead of conducting a separate re-contest.

Requests for a re-contest/re-examination must be submitted through the designated portal within the prescribed timeline and will be considered only for valid reasons supported by prior approval.

3.2 General Exam Rules (All Exams)

- All exams must be attended in person — no online or remote alternatives
- **University ID card** is mandatory for entry into the examination hall. During the contest, if any learner does not carry their ID card, they will be marked **Absent** for that contest.
- Learners who lost the university ID card, they need to report to the Exam Control room (Details will be shared through exam instructions).
- Be seated **15 minutes** before the exam starts, as per the seating plan shared with you in advance.
- Leave all bags, books, mobile phones, and smartwatches outside the hall — even switched off, a phone in your possession counts as UFM
- You cannot leave the exam hall without the invigilator permission, even if you've finished
- No sharing of pens, pencils, calculators, or any material with other students
- Fill in all required details on your answer booklet's cover page accurately
- Sign and number all answer sheets before submission; nothing may be taken out of the hall

Additional Rules for System-Based (Athena) Exams

- Keep your laptop screen brightness above 30% throughout the exam
- No earphones or headphones allowed
- No exchanging laptops or systems with anyone
- No switching tabs or windows, and no use of VPN or proxy software — both are treated as UFM
- If you leave without formally submitting your exam on the system, you will be marked absent and it will be treated as UFM — so always confirm your submission and logout before leaving
- The use of external pre-solution libraries or AI applications is not permitted. The Backend Proctoring Team will review all examinations after completion as well. If any learner is found using unauthorized libraries or AI applications in any manner, it will be treated as UFM. The Proctoring Team may raise concerns even after the examination has been completed.
- Any external digital help/ application not specifically allowed by respective course faculty or mentioned in exam instruction will be considered as prohibited and will come under UFM.

3.3 End-Semester Examination Rules

- Only students with 75% or above attendance (or approved leeway to 50% minimum) can appear — **debarred** students are identified in advance and not allowed near the examination halls.
- Missing the final exam typically results in automatic **Fail** for that course
- There is no re-exam for end-semester — missing an exam means zero for that assessment
- UFM cases in End-Semester exams are reported to the COE; the UFM Committee investigates and the Examination Disciplinary Committee (EDC) reviews and recommends action.
- Students found guilty of UFM in End-Semester exams may face zero marks, course cancellation, or more severe penalties for repeat offences

3.4 Missed an Exam? Re-Contest / Re-Exam Process

If you missed a Weekly Contest or Mid-Term for a genuine reason — medical emergency, family crisis, or representing the university in sports/events — you can apply for a re-exam. Here's how:

- Submit a written request with supporting documents (e.g., medical certificate) to the Academic Operations Team within **48 hours** of the missed exam. **The application process will be shared separately.**
- The approving authority will review the request and verify documents and respond within **5 days**.
- The Director of Academics (or COE for End-Semester related cases) gives final approval or rejection within 3 working days
- If approved, the re-exam is scheduled and the eligible learner list is published — all exam rules apply to the re-exam

Unfair Means (UFM) Policy

Unfair means refers to any act of dishonesty during examinations or assessments. The university takes this extremely seriously. Penalties escalate with each offence and can result in suspension or termination from the program in severe cases.

4.1 What Counts as UFM?

Category	Examples
Unauthorized Materials	Notes, books, written formulas on hands/body/desk, unauthorized marks on clothing, notes hidden in washrooms
Electronic Devices	Carrying a mobile phone (even switched off), smartwatch, earphones, or any gadget not explicitly permitted
Communication	Talking to another student, passing notes, gesturing to share answers, receiving help from outside the hall
Copying	Looking at another student's answer sheet, allowing someone to copy from yours
Destruction of Evidence	Swallowing or hiding chits or materials when caught
Digital Cheating (Mainly in system based exams)	Using pre-stored libraries, unwanted applications, VPN, AI applications
Plagiarism & AI Misuse	Submitting AI-generated work as your own, copying from others' assignments without citation

4.2 UFM Penalties: Continuous (contest) / Internal Assessments

For quizzes, class tests, contests, mid-semester exams, and assignments:

Offence Number	Penalty	Notes
1st Offence	50% marks deducted for that specific assessment	Applies only to the assessment where UFM occurred

2nd Offence	Zero marks for that assessment	Zero marks will be assigned as per weightage of the assessment.
3rd Offence	Grade capped at 'B' for the entire subject	Even if you score higher, your final grade will not exceed B
4th Offence	Zero marks for the entire subject — must reappear or repeat	Subject to reattempt fees; transcript impact

4.3 UFM Penalties: End-Semester / Final Exams

- Invigilators report the case to COE/OAA immediately
- Student is called before the final decision by the Examination Disciplinary Committee (EDC) and given a chance to present their case verbally and in writing
- If possession of unauthorized materials: a new answer sheet is issued and the original sheet is attached to the UFM form
- EDC recommendations may include: marks deduction, zero marks for the subject, or termination from the program for repeated offences

4.4 UFM Penalties Under the Academic Integrity Policy (During Internal Assessment)

The Discipline Ordinance 2025 adds the following penalties for academic misconduct:

Type of Misconduct	Penalty Range	Severe Cases
Cheating in Exams	F grade in course, debarment from next exam	Semester suspension for severe cases; rustication for impersonation
Plagiarism (40–60%)	F in course + probation	Above 60%: suspension or forced course repeat
Plagiarism (25–40%)	Zero for assignment + integrity workshop	Repeat offence: one semester suspension
Plagiarism (10–25%)	Revision + warning	Counselling required
AI Misuse / Unauthorised Collaboration	Warning, resubmission, or F in course	For deliberate misuse: F grade
Falsification / Fake Medical Docs	Suspension, probation, academic penalties	Debarment from entire semester

Group Assessment Note: All members of a group can be penalised if any member commits plagiarism or submits shared work without proper individual contribution.

[Reference: RU Unfair Means Policy \(Ref. RU/OAA/UFM/2024-25/160\); Discipline Ordinance 2025, Chapter 3, Academic Integrity Policy](#)

Discipline & Student Conduct

Rishihood University expects every student to uphold the highest standards of personal conduct, both on and off campus. The Discipline Ordinance applies to all enrolled students and covers all campus spaces, digital platforms, and off-campus university activities.

6.1 Core Values Expected of Every Student

Value	What It Means in Practice
Integrity	Honesty in all academic work, interactions, and self-representation
Respect	For peers, faculty, staff, diversity, culture, and community
Responsibility	Accountability for your own behaviour and decisions
Professionalism	Maturity and conduct appropriate to a university environment
Compassion	Sensitivity to mental health, emotional well-being, and dignity of others

6.2 Prohibited Behaviours

The following are absolutely prohibited on campus, in hostels, and in any university-linked activity:

Zero Tolerance — Severe Misconduct (Applies On & Off Campus):

- Ragging in any form — physical, verbal, or online
- Sexual harassment or assault
- Consumption, possession, or distribution of alcohol, drugs, or tobacco on campus
- Violence, physical assault, or threatening behaviour
- Major theft or destruction of university property
- Mob agitation, inciting unrest, or organized disruption
- Impersonation — academic or otherwise

6.3 Levels of Misconduct

Level	Category	Examples
Level 1	Minor Misconduct	Noise disturbance, minor disobedience, late submissions
Level 2	Moderate Misconduct	Hostel violations, unauthorized visitors, repeat minor offences, cyberbullying
Level 3	Major Misconduct	Academic cheating, vandalism, non-vegetarian food on campus
Level 4	Severe Misconduct	Ragging, sexual harassment, substance use/possession, violence, major theft, mob agitation

6.4 Penalty Matrix

Severity	Possible Penalties
Minor (Level 1)	Warning, counselling, reflection paper, confiscation, temporary restrictions
Moderate (Level 2)	Fine ₹500–₹10,000, community service, hostel probation, academic probation
Major (Level 3)	Fine ₹20,000–₹50,000, suspension (hostel/academic), F grade in course, restriction from events, withholding results
Severe (Level 4)	Semester suspension, hostel expulsion, year-long rustication, expulsion from university, police involvement

6.5 Digital Conduct

Digital misconduct is treated with the same seriousness as physical misconduct:

- No cyberbullying, trolling, or harassing messages
- No posting of defamatory, obscene, or hateful content about anyone
- No sharing of private photos, videos, or chat screenshots without consent
- No impersonation of faculty, staff, or fellow students online
- No sharing of exam content, question papers, or academic materials externally
- The digital policy applies even on personal devices if the content affects an RU student, RU's reputation, or is done on campus

6.6 Discipline Inquiry Process

- Misconduct is reported to the Proctorial Office
- Preliminary assessment is conducted to gauge nature and severity
- If warranted, referred to the University Disciplinary Committee (UDC)
- Student receives a written notice with allegations, evidence, and hearing date
- Student may submit a written explanation and appear before the committee
- Final decision is communicated formally; may be appealed within 7 working days